

algorytm Dijkstry z kolejką priorytetową

```
global Inf, DVGr
global Solved, Qu, n
Inf=999

#DVGr[i]=[[ojciec,value],
           #[1syn,odl],[2syn,..

DVGr=[[[-1,0],[1,3],[4,3]],
       [[None,Inf],[2,1],[0,3]],
       [[None,Inf],[3,3],[5,1]],
       [[None,Inf],[1,3]],
       [[None,Inf],[5,3]],
       [[None,Inf],[0,6],[3,1]]]

Solved=[-1,0]; Qu=[0]; n=0

#skrót zapisu
def vaL(ix):
    return DVGr[ix][0][1]

def Value(pa): #przelicz wartości potomnych
    global n
    for son in DVGr[pa]:
        sonID = son[0]
        if sonID not in Solved :
            if vaL(pa)+ son[1] < vaL(sonID) :
                DVGr[sonID][0][1] = vaL(pa)
                                + son[1]
                DVGr[sonID][0][0] = pa
            if sonID not in Qu :
                Qu.append(sonID)
                n += 1; upHeap(n)
            else: upHeap(Qu.index(sonID))

def minNode(): #wybór najbliższego z kolejki
    Nmin = delMin()
    Solved.append(Nmin)
    return Nmin

Value(0)
while len(Qu) > 1:
    Value(minNode())
```

kolejka priorytetowa Heap

```
def upHeap(k):  
    ix = Qu[k]  
    v = vaL(ix)  
    q = k//2  
    while q>0 and vaL(Qu[q])> v:  
        Qu[k] = Qu[q]  
        k = q; q = q//2  
    Qu[k] = ix
```

```
def delMin():  
    global n  
    dmin = Qu[1]  
    Qu[1] = Qu[n]; n -= 1  
    if n > 1 :  
        downHeap(1,n)  
    del Qu[n+1]  
    return dmin
```

```
def downHeap(k,n):  
    ix = Qu[k]  
    v = vaL(ix)  
    while k <= n//2 :  
        j = 2*k  
        if j < n and vaL(Qu[j]) >  
            vaL(Qu[j+1]):  
            j += 1  
        if v <= vaL(Qu[j]): break  
        Qu[k] = Qu[j]  
        k = j  
    Qu[k] = ix
```